自然语言处理

第11讲：大语言模型的推理部署

杨沐昀

哈尔滨工业大学计算学部
语言技术研究中心

yangmuyun@hit.edu.cn

主要内容

- 大语言模型的算力基础
  - 国内外主流芯片
  - 算力特性
- 推理部署框架
- 大模型压缩技术

11.1 大语言模型的算力基础

- 大语言模型的训练与推理对计算性能要求极高，硬件支持是 AI 技术落地的关键支撑。
- 本节介绍主流 AI 芯片类型，分析其算力特性，为大模型的硬件选型与优化提供依据。
- 理解芯片算力特性可提升大模型训练效率、降低部署成本，适配不同场景需求（如云端训练、边缘推理）。

11.1.1 国内外主流芯片

- 主流 AI 芯片类型：
  - 中央处理器（CPU）：通用计算核心，负责统筹调度
  - 图形处理器（GPU）：并行计算能力强，AI 训练 / 推理主力
  - 张量处理器（TPU）：专为深度学习定制的 ASIC 芯片
  - 神经网络处理器（NPU）：针对神经网络优化的专用芯片

11.1.1 国内外主流芯片

- 中央处理器（CPU）:不含通用CPU
- **Nvidia Grace CPU**
  - 超级芯片设计，高带宽连接
  - 基于 Arm 架构，高性能内核与高效内存子系统
  - 支持 AI 辅助计算：高性能计算和巨型 AI 工作负载
- **AMD DirectML ROCm**
  - 开放式软件栈（驱动、工具、API），支持跨平台兼容
  - 集成 TensorFlow、PyTorch 等主流 AI 库
  - 异构计算优势，适用于 CPU 与 GPU 协同工作场景

11.1.1 国内外主流芯片

- 图形处理器（GPU）
- **Nvidia GPU**
  - 架构迭代：从 Tesla 到 Blackwell系列（10次迭代）
  - 代表系列：
    - GeForce（消费级）、Tesla（高性能计算）、DGX（数据中心）
  - 核心特点：
    - 集成高性能计算核心，高带宽内存接口
    - 支持 CUDA、TensorRT 等优化框架，FP16/INT8/TF32 多精度计算
    - 生态成熟，主导 AI 训练与推理市场

11.1.1 国内外主流芯片

- 图形处理器（GPU）
- **AMD GPU**
  - 架构演进：GCN→RDNA
  - 代表系列：Radeon（消费级）、Radeon Pro（专业级）、Instinct（AI 计算，如 MI25、MI60和MI325X）
  - 核心特点：
    - 多级缓存系统，高速内存接口
    - 支持 OpenCL、HIP 编程模型，适配部分 AI 框架

11.1.1 国内外主流芯片

- 张量处理器（TPU）——**Google TPU**
  - 专为 TensorFlow 框架设计的 ASIC 芯片
  - 技术特点：
    - 高并行性，擅长矩阵运算，支持混合精度计算
    - 内置大量片上存储，减少对外部内存依赖
    - 低功耗设计，适合大规模分布式训练
  - 版本迭代：
    - v1（2015）→v2（2017）→v3（2019）→v4（2021）→Trillium（2024）

11.1.1 国内外主流芯片

- 神经网络处理器（NPU）
  - ARM架构
  - 这里讨论的不包括推理专用NPU
- **华为昇腾 910B3**
  - 架构：达芬奇架构，集成数千个处理核心
  - 特点：
    - 支持高效并行计算、多精度计算
    - 融合片上异构处理与高速互连技术
    - 全场景适配（训练、推理、边缘计算），兼容 MindSpore 框架

11.1.1 国内外主流芯片

- 神经网络处理器（NPU）
- **寒武纪 NPU**
  - 代表产品：寒武纪 1A/1H/1M、MLU270/290
  - 特点：
    - 初期面向终端设备
    - 低功耗、高效率，超越传统 CPU/GPU 的神经网络处理能力
    - 支持 Caffe、TensorFlow 等框架，适用于终端设备（手机、无人机）与行业场景（智能驾驶、CV）

11.1.2 算力特性

- 芯片算力特性分析维度
  - 版本与更新时间：芯片型号迭代周期，技术更新速度
  - 配置难度：硬件部署、软件适配的复杂度（生态成熟度影响）
  - 功能丰富度：支持的计算精度、框架兼容性、应用场景覆盖范围
    - 重点考虑对AI模型训练和推理的支持能力

算力特性对比（一）

芯片类型	代表产品	版本与更新时间	配置难度	功能丰富度
CPU	Nvidia Grace	持续更新，适配 AI 工作负载	低（兼容主流系统）	通用计算能力强，支持多任务处理，与 GPU 协同优化
	AMD ROCm	ROCm 6.1.3（2024.6），定期更新	高（开源需自行配置）	跨平台兼容，集成主流 AI 库，异构计算优势显著
TPU	Google TPU v5e	V1(2015)→Trillium(2024)，与 TensorFlow 同步	中（需 Google Cloud 访问）	专为深度学习优化，大规模训练性能优越，对 TensorFlow 支持极佳，其他框架有限

算力特性对比（二）

芯片类型	代表产品	版本与更新时间	配置难度	功能丰富度
GPU	Nvidia A100/H100	A100（2020）、H100（2022），迭代快	低（CUDA 生态完善）	支持多精度计算，兼容所有主流 AI 框架，覆盖训练 / 推理全场景
	AMD MI25/MI60	MI100（2020）、MI200（2021）	中（ROCm 生态待成熟）	图形与计算能力均衡，支持 OpenCL/HIP，适合特定 AI 算法优化
NPU	华为昇腾 910B3	2019 年发布，持续更新软件工具	中（需适配 MindSpore）	全场景 AI 支持，端到端解决方案，适合云端与边缘计算
	寒武纪 MLU290	迭代较慢，专注推理 / 训练	高（需专业领域知识）	深度学习能效比高，支持主流框架，适用于终端设备与行业定制场景

11.1 大语言模型的算力基础

- 小结：各芯片适用场景
  - Nvidia GPU：AI 领域最常见解决方案，尤其适合大模型训练与通用推理，生态成熟。
  - AMD GPU：性价比优势，适合预算有限或异构计算场景，依赖 ROCm 优化。
  - Google TPU：TensorFlow 框架专属，适合谷歌生态下的大规模分布式训练。
  - 华为昇腾 / NPU：国产替代核心，适配国内 AI 框架（如 MindSpore），覆盖全场景。
  - CPU：辅助角色，负责任务调度，与 GPU/NPU 协同提升系统效率。

主要内容

- 大语言模型的算力基础
- 推理部署框架
  - 主流推理部署框架介绍
  - 推理部署框架性能对比
- 大模型压缩技术

11.2 推理部署框架

- 大语言模型运行需满足应用需求
  - 参数规模庞大，运行成本高昂且速度缓慢，需要适当的优化技术
- 大模型推理部署框架奠定了其实际应用基础
  - 简化了模型的部署过程
  - 大幅降低运行所消耗的资源
  - 而且还提高了运行速度
  - 在各种不同的计算环境中高效运行，降低使用者的成本

11.2.1 主流推理部署框架介绍

- vLLM：一个快速且易于使用的开源库，专为大语言模型的推理和部署而设计
- vLLM的主要特性
  - 使用PagedAttention对注意力键和值的内存进行高效地管理
  - 对传入的请求进行连续批处理
  - 与流行的HuggingFace模型无缝集成
  - 支持张量并行和流水线并行以进行分布式推理
  - 使用各种解码算法（包括并行采样、束搜索等）进行高吞吐量服务

11.2.1 主流推理部署框架介绍

- Ollama
- 一个简化大语言模型本地部署和运行过程的工具
  - 开发者可以访问和运行一系列预构建的模型，或者导入和自定义自己的模型，无需关注复杂的底层实现细节
- 运行模型：Ollama run model
  - 运行本地“model”
  - 自动拉取“model”到本地运行
  - 提供交互界面，通过命令行输入问题或者提示即可

11.2.1 主流推理部署框架介绍

- HuggingFace TGI (Text Generation Inference)
- 一种用于部署和提供大语言模型的工具包
  - 为最流行的开源大语言模型提供高性能的文本生成服务
  - 部署和运行大语言模型的一个理想选择
- 关键的特性
  - 简单的启动器，可轻松部署大多数流行的大语言模型
  - 支持多GPU环境下的张量并行，加快推理速度
  - 对传入的请求进行连续批处理，提高总体吞吐量
  - 流行架构上应用Flash Attention等技术优化推理，提高性能
  - 支持使用Safetensors进行权重加载，增强数据安

11.2.1 主流推理部署框架介绍

- TensorRT-LLM
- 加速和优化大语言模型在NVIDIA GPU上推理的开源库
- 主要特性
  - 一个简单、开源的Model Definition API：集成了预处理和后处理步骤，以及多GPU多节点通信原语
    - GPU上的LLM推理性能提升巨大
  - In-flight Batching机制：移除一个批次中已完成的推理序列，响应新的请求，减少队列等待
  - 有GPU的支持，包括NVIDIA Hopper、NVIDIA Ada Lovelace、NVIDIA Ampere、NVIDIA Turing和NVIDIA Volta架构

11.2.1 主流推理部署框架介绍

- DeepSpeed-MII（Model Implementations for Inference）
  - 开源的Python库：低延迟/低成本推理+易于访问
- 主要特性
  - 支持数千种广泛使用的深度学习模型
  - 利用了DeepSpeed-Inference的系列推理优化技术：例如用于transformer的deepfusion、用于多GPU推理的自动张量切片、即时量化（ZeroQuant）以及一些其他的优化
  - 支持低成本部署：几行代码实现本地或者云部署

11.2.1 主流推理部署框架介绍

- llama.cpp
- 主要目标：大语言模型推理部署在各种硬件上（本地和云端），最少的设置+最优的性能
- 核心特点
  - 纯C/C++实现：不需要外部依赖，使得框架更加轻量化，易于编译和部署
  - 量化技术：支持1.5-bit、2-bit、3-bit、4-bit、5-bit、6-bit和8-bit的整数量化技术，用于加快推理速度并减少内存使用
  - 混合推理：支持CPU+GPU混合推理，可以在总显存容量不足以完全加载模型时，部分加速模型的运行

11.2.2推理部署框架性能对比

- 解码速度：每秒能够处理的词元（token）数量
  - 衡量模型生成文本或处理任务快慢的重要参考指标
  - 对于实时应用或需要快速响应的应用场景尤为重要
- 显存占用： GPU专用内存
  - 主要用于存储模型的权重、激活值以及其他计算过程中产生的中间数据
  - 决定了模型能否成功加载到GPU上，从而影响运行速度
- 内存占用
  - 计算机存储正在运行的应用程序数据
  - 影响系统响应速度以及能否加载并运行模型

11.2.2推理部署框架性能对比

- Llama3-8B-Instruct模型：以‘The future of AI is’作为提示词进行文本生成

框架 / 参数	机构	生成词元数	解码速度/(token/s)	占用显存/GB	占用内存/GB
base:Llama3-8B-Instruct	—	517	5.76	18.2	1.5
vLLM	加州大学伯克利分校	16	17.3	17.8	15.9
Ollama	Ollama	493	20.4	16.1	11.3
HuggingFace TGI	HuggingFace	100	23.9	17.3	23.1
TensorRT-LLM	NVIDIA	188	12.7	12.8	9.1
DeepSpeed-MII	Microsoft	280	38.0	22.8	17.15
Llama.cpp	Georgi Gerganov	585	11.7	17.0	16.8

主要内容

- 大语言模型的算力基础
- 推理部署框架
- 大模型压缩技术
  - 模型量化
  - 模型蒸馏

11.3大模型压缩技术

- 低资源部署问题
  - 实际应用中计算资源有限的场景
- 大模型压缩技术：降低大语言模型的部署成本，并且提升模型的推理性能
  - 模型量化
  - 模型蒸馏

11.3.1模型量化

- 大模型的量化：是指将浮点型参数（通常为float32）映射为整型参数（通常为int8）的过程
  - 用更少的位数来表示模型的参数
  - 大幅减少存储需求和计算成本
- 量化对象
  - 模型权重（weight）量化
    - 模型存储=模型参数量*数值精度（数据类型）
    - 将较高精度表示的权重转换为使用较少比特数的数据类型
  - 模型激活值（activation）量化
    - 针对每一层输出的结果，即激活值

11.3.1模型量化

例子？

- 均匀量化：q为量化后的整数

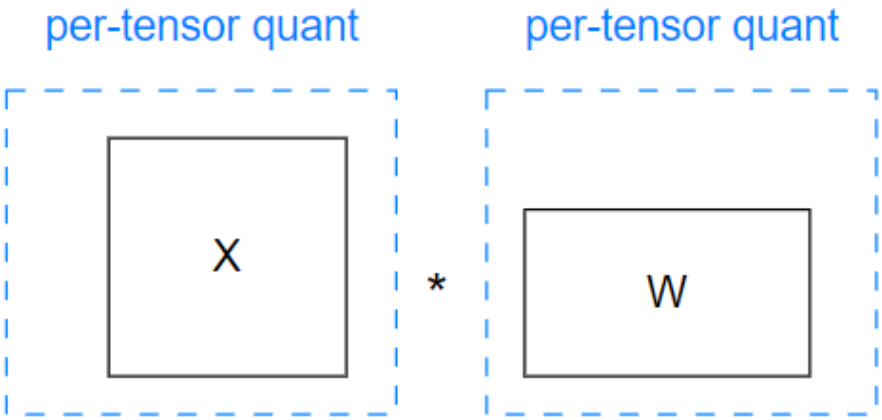
$$q = \text{clip}\left(\text{round}\left(\frac{r}{s} + z\right), q_{\min}, q_{\max}\right)$$

- r：量化前的浮点数
- S：数据量化的缩放因子
  将浮点数映射到整数范围
- Z：零点因子，用于调整数据量化的偏移
- $round(\cdot)$：取整操作
- $clip(\cdot)$：对数据的截断操作
- q_{min}和q_{max}：量化后的最小值和最大值，
- (q_{min}, q_{max})：量化网格限制，r值被裁剪到该范围内

$$s = \frac{\max - \min}{\text{int_max} - \text{int_min}}$$

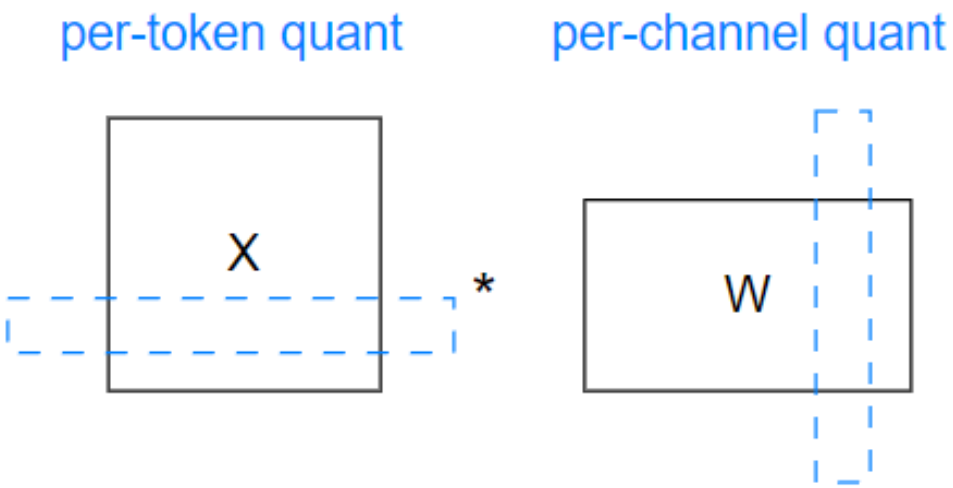
11.3.1模型量化

- 量化策略：针对的不同维度
- 逐层量化（per-tensor）
  - 对模型的每一层整体进行统一的量化操作
  - 每一层的权重和激活值使用相同的量化参数



11.3.1模型量化

- 量化策略：针对的不同维度
- 逐层量化（per-tensor）
- 逐通道量化（per-token&per-channel）
  - 对神经网络中每个通道的权重或激活值进行单独量化
  - 每个通道都有其独立的量化参数



11.3.1模型量化

- 量化策略：针对的不同维度
- 逐层量化（per-tensor）
- 逐通道量化（per-token&per-channel）
- 逐组量化（per-group）
  - 介于逐层量化和逐通道量化之间的量化方法
  - 针对模型的特定层或特征图进行分组处理
  - 通过将多个通道组合在一起进行量化，从而在降低计算和存储成本的同时，尽量保持模型的性能
  - 当group_size=1时，逐组量化和逐层量化等价；
  - 当group_size=num_filters时，逐组量化和逐通道量化等价。

11.3.1模型量化

- 应用量化压缩模型放法：量化感知训练（QAT）
- 在“训练过程中”考虑量化影响的技术，旨在提高量化后模型的性能与精度。
- 在训练过程中，使用近似量化函数来模拟权重和激活值的量化
  - 在前向传播时，模型会将浮点数量化为整数，并在计算中使用这些量化值
  - 可以采用伪量化的方法，即在反向传播时仍使用浮点数，但在前向传播中使用量化后的值
- 帮助模型在训练阶段适应低精度表示，增强模型处理因量化而造成的精度损失的能力

11.3.1模型量化

- 应用量化压缩模型放法：量化感知微调（QAF）
- 在“微调过程中”对模型进行量化，旨在确保经过微调的模型在低精度下仍然能保持性能
- 通过将量化感知过程整合到微调中，可以让微调后的模型同时实现压缩空间和保持性能。

11.3.1模型量化

- 应用量化压缩模型放法：训练后量化（PTQ）
- 在训练后对已经训练/微调好的模型进行量化，目的在于减少模型的存储和计算复杂度
  - 优点：实现简单且计算效率高
  - 缺点：在量化过程中会引入一定程度的精度损失。
- 这一方法更容易应用到实际中
- 主流方法
  - LLM.int8()
  - GPTQ
  - SmoothQuant
  - SpQR
  - AWQ

11.3.1模型量化

- 训练后量化（PTQ）:LLM.int8()
  - 设计思路：对离群点所在的特征进行单独计算，对剩余的特征进行量化。
  - 步骤1：对输入的隐藏状态按列提取出异常值（即离群点所在的特征）
  - 步骤2：对离群点所在的特征保持float16精度进行矩阵运算，对剩余的特征进行int8量化，对量化后的特征进行矩阵运算
  - 步骤3：对非离群点所在特征的矩阵计算结果进行反量化，和离群特征矩阵运算结果相加，得到最后的精度为float16的输出。

11.3.1模型量化

- 训练后量化（PTQ）：GPTQ
  - 原理来自 OBQ<-OBS<-OBD
  - 思路：通过数学计算得到每个参数对目标函数的影响
  - 方法：通过逐层量化解决每层的重构问题
  - 目标是找到一组量化后的权重 Wˆ，使得量化后的层输出与全精度层输出之间的误差最小化。

$$\operatorname{argmin}_{\hat{W}} \|WX - \hat{W}X\|_2^2$$

11.3.1模型量化

- 训练后量化（PTQ）:SmoothQuant
  - 一种在保证准确率的同时可以进行高效推理的训练后量化方法。
  - 观察：激活值 X 难以量化， 因为离群点拉伸了量化范围，导致大多数值的有效位数减少。
  - 方法：引入平滑因子 s 来平滑激活值中的异常值， 通过数学上的等效变换将量化的难度从激活值转移到权重上
    - （激活值， 权重： 难量化， 非常容易量化） → （激活值，权重： 较容易量化， 较容易量化）

11.3.1模型量化

- 训练后量化（PTQ）：SpQR（Sparse-Quantized Representation）
  - 首次实现了大语言模型跨模型规模的近乎无损压缩，同时达到了与先前方法相似的压缩水平
  - 设计思路：识别并保护这些导致特别大量化误差的异常值权重，并对这些权重保持高精度存储，同时将其他权重量化至3-4位
  - 实现方法
    - 使用小部分校准输入X在每层评估敏感程度
    - 权重矩阵W中某个权重w_ij的敏感程度s_ij：原始预测与所有该权重被量化后的权重矩阵W'预测之间的最小差值
    - 对量化权重影响最大的取top 1%参数以稀疏矩阵的形式存储保护
    - 对剩下的权重进行量化。

11.3.1模型量化

- 训练后量化（PTQ）：AWQ（Activation-aware Weight Quantization）
  - 设计思路：保护少量的关键权重就可以大幅度减少量化误差
  - 识别和保护关键权重的实现方式与SpQR不同
    - SpQR基于对误差的计算找到关键权重，并将top 1%用稀疏矩阵进行存储
    - AWQ则是基于激活值和权重的大小进行缩放保护，以避免单独保存关键权重的方式效率低且实现复杂。
    - 实现方式类似SmoothQuant，即寻找一个缩放因子s，让激活值除以s（缩小变化范围，降低离群点的影响）；让权重乘s，平衡激活值的变化。

11.3.2 模型蒸馏

- 大模型的蒸馏：是指将一个复杂的、庞大的教师模型的知识转移到一个更小、更高效的学生模型的过程
- 核心思想：让学生模型学习教师模型的泛化能力，而不是单纯地对训练数据进行拟合
- 目的：减少计算资源和内存需求的同时，能够有效地保持甚至接近教师模型的性能。

11.3.2 模型蒸馏

- 一个知识蒸馏过程，即知识转移
  - 学生模型通过学习教师模型的输出（通常是soft label或概率分布）来实现的。
- 损失函数：蒸馏损失、教师损失和学生损失共同构成
  - 教师损失：评估教师模型在原始任务上的性能，通常不直接影响学生模型的训练
  - 学生损失：评估学生模型在原始任务上的性能，是学生模型训练的一部分
  - 蒸馏损失：评估学生模型的输出和教师模型的输出之间的差距

$$L_{distill} = -\frac{1}{T}\sum_{i=1}^{N} y_i^{teacher} \log(y_i^{student})$$

11.3.2 模型蒸馏

- 损失函数
  - 综合损失函数

$$L_{\text{total}} = \alpha L_{\text{distill}} + (1-\alpha) L_{\text{student}}$$

  - α是一个超参数，用于平衡蒸馏损失。

11.3.2 模型蒸馏

- 知识蒸馏方法：标准知识蒸馏
  - 类似于传统的知识蒸馏方法，教师换成大语言模型
  - 典型的方法MINILLM：白箱方法
    - 可以访问教师模型的内部结构和中间层输出
    - 通过最小化反向KL散度来蒸馏知识
  - 典型的方法GKD：黑箱方法
    - 只能访问教师模型的输入和最终输出
    - 对学生的输出序列进行采样来处理分布不匹配的问题
    - 以优化替代散度来解决模型的不足

11.3.2 模型蒸馏

- 知识蒸馏方法：基于涌现能力的知识蒸馏
  - 目标学习教师模型的常识和涌现能力
  - 大模型的涌现能力是指大型语言模型在训练过程中自然发展出的、超出初始设计和预期的能力
    - 这些能力通常在模型规模达到一定程度时才会显现出来，而且往往不是通过显示的设计或训练目标直接获得的
    - 大模型的涌现能力包括：上下文学习（In-Context Learning，ICL）、思维链（Chain of Thought，CoT）和指令遵循（Instruction Following，IF）。

结语

- 大语言模型推理部署阶段设计的
  - 算力
  - 推理框架
  - 模型量化
  - 模型蒸馏